

Вот подробный конспект семинара по Модулям 5-6. Структура адаптирована под конвертацию в `.ipynb`: теория чередуется с кодовыми ячейками, каждое задание сопровождается примером решения.

# Семинар: Модули 5-6. Dependency Injection и асинхронные базы данных

## Часть 1. Dependency Injection в FastAPI

### 1.1. Проблема: жесткая связанность

Когда endpoint напрямую создает зависимости, код становится неподдерживаемым:

```
In [1]: # ПЛОХО: жестко зашито всё
@app.get("/users/{user_id}")
async def get_user(user_id: int):
    db = AsyncPostgresConnection(host="localhost", port=5432)
    await db.connect()
    user = await db.fetchrow("SELECT * FROM users WHERE id = $1", user_id)
    await db.close()
    return dict(user)
```

Проблемы:

1. Невозможно протестировать без настоящей БД.
2. Дублирование кода подключения в каждом endpoint.
3. Невозможно сменить реализацию (PostgreSQL -> SQLite для тестов).
4. Нет управления ресурсами: при исключении `db.close()` может не выполняться.

### 1.2. Инверсия управления (IoC)

**Голливудский принцип:** "Не звоните нам, мы сами вам позвоним".

**Традиционный код:**

```
In [2]: Endpoint -> создает Database -> создает Connection -> выполняет запрос
```

## IoC:

In [3]: Framework -> создает Database -> передает в Endpoint -> Endpoint использует

Управление перевернуто: инфраструктура "подставляет" себя в бизнес-логику.

### Три способа внедрения зависимостей:

| Способ            | Описание                 | Пример                                   |
|-------------------|--------------------------|------------------------------------------|
| Через конструктор | Зависимости при создании | <code>service = UserService(repo)</code> |
| Через сеттер      | После создания           | <code>service.set_repo(repo)</code>      |
| Через интерфейс   | В метод при вызове       | <code>def method(self, repo)</code>      |

FastAPI использует вариант через интерфейс (параметры функции), но автоматизирует его.

## 1.3. Система Depends: базовый синтаксис

```
In [4]: from fastapi import FastAPI, Depends

app = FastAPI()

async def common_parameters(q: str | None = None, skip: int = 0, limit: int = 10
0):
    return {"q": q, "skip": skip, "limit": limit}

@app.get("/items/")
async def read_items(common: dict = Depends(common_parameters)):
    return common

@app.get("/users/")
async def read_users(common: dict = Depends(common_parameters)):
    return common
```

Что происходит:

1. FastAPI видит `common: dict = Depends(common_parameters)` .
2. Вызывает `common_parameters(q=..., skip=..., limit=...)` , передавая query-параметры.
3. Результат передается в endpoint как `common` .

```
In [5]: # Классы-зависимости
class PaginationParams:
    def __init__(self, skip: int = 0, limit: int = 100):
        self.skip = skip
        self.limit = limit

@app.get("/items/")
async def read_items(pagination: PaginationParams = Depends()):
    return {"skip": pagination.skip, "limit": pagination.limit}
```

## 1.4. Вложенные зависимости

Зависимости могут зависеть от других зависимостей. FastAPI разрешает дерево рекурсивно.

```
In [6]: from fastapi import FastAPI, Depends

app = FastAPI()

async def get_query_token(token: str):
    return token

async def get_token_header(token: str = Depends(get_query_token)):
    return {"token": token}

@app.get("/items/")
async def read_items(info: dict = Depends(get_token_header)):
    return info

# Дерево разрешения:
# read_items
#   └─ get_token_header
#       └─ get_query_token
```

## 1.5. Кэширование зависимостей

По умолчанию `Depends` кэширует результат в рамках одного запроса:

```
In [7]: async def get_db():
        db = {"connection": "active", "id": id(object())}
        try:
            yield db
        finally:
```

```

        db["connection"] = "closed"

@app.get("/test/")
async def test_cache(db1 = Depends(get_db), db2 = Depends(get_db)):
    assert db1 is db2 # True! Один и тот же объект
    return {"same": db1 is db2}

# Отключение кэширования (редко нужно):
# db = Depends(get_db, use_cache=False)

```

## 1.6. Управление жизненным циклом: yield

Самый мощный паттерн — генераторы с `yield`. Это асинхронный контекстный менеджер, встроенный в DI.

```

In [8]: from fastapi import FastAPI, Depends

app = FastAPI()

async def get_resource():
    resource = {"status": "created", "data": []}
    print("SETUP: ресурс создан")
    try:
        yield resource
    finally:
        print("TEARDOWN: ресурс освобожден")

@app.get("/use/")
async def use_resource(res: dict = Depends(get_resource)):
    res["data"].append("used")
    return res

# uvicorn seminar_56:app --port 8000
# curl http://localhost:8000/use/
# В консоли: SETUP -> TEARDOWN

```

### Транзакции с rollback при ошибке:

```

In [9]: async def get_db_with_transaction():
        db = {"session": "active", "committed": False}
        try:
            yield db
            db["committed"] = True

```

```

        print("COMMIT")
    except Exception:
        print("ROLLBACK")
        raise
    finally:
        print("CLOSE")

@app.get("/fail/")
async def failing_endpoint(db: dict = Depends(get_db_with_transaction)):
    raise ValueError("oops") # вызовет ROLLBACK, затем CLOSE

```

## 1.7. Зависимости для роутера и приложения

```

In [10]: from fastapi import FastAPI, APIRouter, Depends, HTTPException, Header

async def verify_token(x_token: str = Header()):
    if x_token != "secret-token":
        raise HTTPException(status_code=403, detail="Invalid token")

# На весь роутер
admin_router = APIRouter(
    prefix="/admin",
    dependencies=[Depends(verify_token)]
)

@admin_router.get("/dashboard")
async def admin_dashboard():
    return {"status": "ok"}

@admin_router.get("/stats")
async def admin_stats():
    return {"users": 1000}

# На всё приложение
app = FastAPI(dependencies=[Depends(verify_token)])
app.include_router(admin_router)

```

## 1.8. Паттерн Repository

Repository — абстракция, скрывающая детали доступа к данным. Бизнес-логика не знает о SQL.

```

In [11]: from typing import Protocol
          from pydantic import BaseModel

          class User(BaseModel):
              id: int
              name: str
              email: str

          class UserRepository(Protocol):
              async def get_by_id(self, user_id: int) -> User | None: ...
              async def create(self, user: User) -> User: ...

          # In-memory реализация для тестов
          class InMemoryUserRepository:
              def __init__(self):
                  self._users: dict[int, User] = {}
                  self._counter = 1

              async def get_by_id(self, user_id: int) -> User | None:
                  return self._users.get(user_id)

              async def create(self, user: User) -> User:
                  user.id = self._counter
                  self._users[user.id] = user
                  self._counter += 1
                  return user

          # Зависимость
          async def get_user_repo() -> UserRepository:
              return InMemoryUserRepository()

          @app.get("/users/{user_id}")
          async def get_user(user_id: int, repo: UserRepository = Depends(get_user_repo)):
              user = await repo.get_by_id(user_id)
              if not user:
                  raise HTTPException(status_code=404, detail="User not found")
              return user

          @app.post("/users/")
          async def create_user(user: User, repo: UserRepository = Depends(get_user_repo)):
              return await repo.create(user)

```

## 1.9. Паттерн Unit of Work

Unit of Work группирует операции в одну транзакцию.

```

In [12]: class UnitOfWork:
    def __init__(self):
        self._data: dict = {}
        self._committed = False

    async def commit(self):
        self._committed = True
        print("UoW: COMMIT")

    async def rollback(self):
        self._committed = False
        print("UoW: ROLLBACK")

async def get_uow() -> UnitOfWork:
    uow = UnitOfWork()
    try:
        yield uow
        await uow.commit()
    except Exception:
        await uow.rollback()
        raise

@app.post("/orders/")
async def create_order(uow: UnitOfWork = Depends(get_uow)):
    # Имитация создания заказа
    return {"status": "created", "committed": uow._committed}

```

## 1.10. Инъекция конфигурации

```

In [13]: from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str = "sqlite:///default.db"
    debug: bool = False

    class Config:
        env_file = ".env"

settings = Settings()

def get_settings() -> Settings:
    return settings

@app.get("/health")

```

```
async def health(settings: Settings = Depends(get_settings)):
    return {"debug": settings.debug}
```

## 1.11. Тестирование зависимостей: dependency\_overrides

```
In [14]: from fastapi import FastAPI, Depends
from fastapi.testclient import TestClient

app = FastAPI()

async def get_db():
    return {"type": "real", "data": "postgres"}

@app.get("/items/")
async def read_items(db: dict = Depends(get_db)):
    return db

# --- Тестирование ---
def override_get_db():
    return {"type": "fake", "data": "in-memory"}

app.dependency_overrides[get_db] = override_get_db

client = TestClient(app)

def test_read_items():
    response = client.get("/items/")
    assert response.status_code == 200
    assert response.json()["type"] == "fake"

# Очистка после тестов
# app.dependency_overrides.clear()
```

### Test Double:

| Тип   | Описание                      | Пример                            |
|-------|-------------------------------|-----------------------------------|
| Dummy | Ничего не делает              | <code>Mock()</code> без настройки |
| Fake  | Рабочая упрощенная реализация | <code>InMemoryRepository</code>   |
| Stub  | Заранее заданные ответы       | <code>lambda: 42</code>           |



| Тип  | Описание                 | Пример                                 |
|------|--------------------------|----------------------------------------|
| Spy  | Запоминает вызовы        | Mock с <code>assert_called_with</code> |
| Mock | Полностью контролируемый | <code>unittest.mock.Mock</code>        |

```
In [15]: from unittest.mock import AsyncMock

def override_get_ml_model():
    mock = AsyncMock()
    mock.predict.return_value = [0.95, 0.05]
    return mock
```

## Часть 2. Асинхронные базы данных

### 2.1. ACID: фундамент надежности

| Свойство           | Суть                              | Пример для ML                                                                   |
|--------------------|-----------------------------------|---------------------------------------------------------------------------------|
| <b>Atomicity</b>   | Все операции или ни одной         | INSERT в <code>predictions</code> + <code>prediction_features</code> — атомарно |
| <b>Consistency</b> | Ограничения не нарушаются         | FOREIGN KEY гарантирует, что фичи привязаны к существующему предсказанию        |
| <b>Isolation</b>   | Параллельные транзакции не мешают | Два запроса на предсказание не перезапишут друг друга                           |
| <b>Durability</b>  | После COMMIT данные навсегда      | WAL: сначала журнал на диск, потом данные                                       |

### 2.2. Уровни изоляции и феномены

**Феномены:**

- 1. **Dirty Read** — чтение незафиксированных данных.
- 2. **Non-Repeatable Read** — повторное чтение дает другой результат.
- 3. **Phantom Read** — повторный запрос с условием возвращает новые строки.
- 4. **Lost Update** — одно обновление перезаписывает другое.

| Уровень          | Dirty Read | Non-Repeatable | Phantom Read |
|------------------|------------|----------------|--------------|
| READ UNCOMMITTED | Да         | Да             | Да           |
| READ COMMITTED   | Нет        | Да             | Да           |
| REPEATABLE READ  | Нет        | Нет            | Нет*         |
| SERIALIZABLE     | Нет        | Нет            | Нет          |

\* В PostgreSQL REPEATABLE READ предотвращает и phantom reads.

### Выбор уровня:

- **READ COMMITTED** (по умолчанию в PostgreSQL): для большинства веб-приложений.
- **REPEATABLE READ**: для расчета агрегатов внутри транзакции.
- **SERIALIZABLE**: для финансовых операций (реализован через SSI в PostgreSQL).

## 2.3. Индексы: B-Tree

В-дерево — самобалансирующееся дерево с высоким фактором ветвления (тысячи ключей в узле).

### Почему не бинарное дерево:

- Дисковые операции дороги (4-10 мс).
- При факторе ветвления 1000 и 1 млн записей высота = 2.
- Узлы соответствуют страницам диска (8 КБ).

### Сложность:

- Поиск:  $O(\log_B n)$
- Вставка:  $O(\log_B n)$
- Удаление:  $O(\log_B n)$

```
In [16]: -- Частичный индекс
CREATE INDEX idx_active_users ON users(email) WHERE is_active = true;

-- Покрывающий индекс
CREATE INDEX idx_users_covering ON users(email) INCLUDE (name, created_at);
```

## 2.4. SQLAlchemy 2.0: асинхронный стиль

## Эволюция:

```
In [17]: # SQLAlchemy 1.x – query-style
users = session.query(User).filter(User.age > 18).order_by(User.name).all()

# SQLAlchemy 2.0 – select-style, unified API
result = await session.execute(select(User).where(User.age > 18).order_by(User.name))
users = result.scalars().all()
```

## Асинхронное ядро:

```
In [18]: from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession, async_sessionmaker

from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
from sqlalchemy import select, insert, update, delete

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str]
    email: Mapped[str]

# Движок (для SQLite async – aiosqlite)
DATABASE_URL = "sqlite+aiosqlite:///./test.db"

engine = create_async_engine(
    DATABASE_URL,
    echo=True,
    pool_size=5,
    max_overflow=10,
)

AsyncSessionLocal = async_sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False,
    autoflush=False,
)

# Создание таблиц
```

```

async def init_db():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)

# Зависимость для FastAPI
async def get_db():
    async with AsyncSessionLocal() as session:
        yield session

# CRUD операции
async def create_user(session: AsyncSession, name: str, email: str):
    user = User(name=name, email=email)
    session.add(user)
    await session.commit()
    await session.refresh(user)
    return user

async def get_user_by_id(session: AsyncSession, user_id: int):
    result = await session.execute(select(User).where(User.id == user_id))
    return result.scalar_one_or_none()

async def update_user(session: AsyncSession, user_id: int, name: str):
    await session.execute(
        update(User).where(User.id == user_id).values(name=name)
    )
    await session.commit()

async def delete_user(session: AsyncSession, user_id: int):
    await session.execute(delete(User).where(User.id == user_id))
    await session.commit()

# Демонстрация
async def demo_crud():
    await init_db()
    async with AsyncSessionLocal() as session:
        user = await create_user(session, "Alice", "alice@example.com")
        print(f"Created: {user.id}, {user.name}")

        found = await get_user_by_id(session, user.id)
        print(f"Found: {found.name}")

        await update_user(session, user.id, "Alice Smith")
        updated = await get_user_by_id(session, user.id)
        print(f"Updated: {updated.name}")

        await delete_user(session, user.id)

```

```

        deleted = await get_user_by_id(session, user.id)
        print(f"Deleted: {deleted}")

import asyncio
asyncio.run(demo_crud())

```

## 2.5. Интеграция с FastAPI: сессия на запрос

```

In [19]: from fastapi import FastAPI, Depends, HTTPException
        from sqlalchemy.ext.asyncio import AsyncSession
        from sqlalchemy import select

        app = FastAPI()

        async def get_db() -> AsyncSession:
            async with AsyncSessionLocal() as session:
                yield session

        @app.get("/users/{user_id}")
        async def get_user(user_id: int, db: AsyncSession = Depends(get_db)):
            result = await db.execute(select(User).where(User.id == user_id))
            user = result.scalar_one_or_none()
            if not user:
                raise HTTPException(status_code=404, detail="User not found")
            return {"id": user.id, "name": user.name, "email": user.email}

        @app.post("/users/")
        async def create_user_endpoint(
            name: str,
            email: str,
            db: AsyncSession = Depends(get_db)
        ):
            user = User(name=name, email=email)
            db.add(user)
            await db.commit()
            await db.refresh(user)
            return {"id": user.id, "name": user.name, "email": user.email}

```

### Unit of Work в FastAPI:

```

In [20]: class DatabaseUnitOfWork:
        def __init__(self, session: AsyncSession):
            self.session = session

```

```

    async def commit(self):
        await self.session.commit()

    async def rollback(self):
        await self.session.rollback()

async def get_uow(session: AsyncSession = Depends(get_db)) -> DatabaseUnitOfWork:
    uow = DatabaseUnitOfWork(session)
    try:
        yield uow
        await uow.commit()
    except Exception:
        await uow.rollback()
        raise

class UserRepository:
    def __init__(self, session: AsyncSession):
        self._session = session

    async def get_by_id(self, user_id: int):
        result = await self._session.execute(select(User).where(User.id == user_id))
        return result.scalar_one_or_none()

    async def create(self, user: User):
        self._session.add(user)
        await self._session.flush()
        return user

async def get_user_repo(session: AsyncSession = Depends(get_db)):
    return UserRepository(session)

@app.post("/users/")
async def create_user(
    name: str,
    email: str,
    repo: UserRepository = Depends(get_user_repo),
    uow: DatabaseUnitOfWork = Depends(get_uow),
):
    user = User(name=name, email=email)
    await repo.create(user)
    # commit произойдет автоматически
    return {"id": user.id, "name": user.name}

```

## 2.6. Deadlock и retry-логика

```
In [21]: import random
import asyncio

# Имитация deadlock
class DeadlockDetectedError(Exception):
    pass

async def transfer_funds(from_id: int, to_id: int, amount: float, max_retries: int = 3):
    for attempt in range(max_retries):
        try:
            # Имитация транзакции
            print(f"Attempt {attempt + 1}: transfer {amount} from {from_id} to {to_id}")

            await asyncio.sleep(0.1)

            # Симулируем deadlock на первых попытках
            if attempt < 2 and random.random() < 0.7:
                raise DeadlockDetectedError("deadlock detected")

            print("Transfer successful")
            return {"status": "success"}

        except DeadlockDetectedError:
            wait_time = 0.1 * (2 ** attempt) + random.uniform(0, 0.1) # exponential backoff + jitter
            print(f"Deadlock! Retrying in {wait_time:.3f}s...")
            await asyncio.sleep(wait_time)

    raise DeadlockDetectedError("Max retries exceeded")

async def demo_deadlock():
    random.seed(42)
    result = await transfer_funds(1, 2, 100.0)
    print(result)

asyncio.run(demo_deadlock())
```

**Оптимистичные блокировки:**

```

In [22]: class Account(Base):
    __tablename__ = "accounts"
    id: Mapped[int] = mapped_column(primary_key=True)
    balance: Mapped[float]
    version: Mapped[int] = mapped_column(default=0)

    async def transfer_optimistic(from_id: int, to_id: int, amount: float, session: AsyncSession):
        for attempt in range(10):
            from_acc = await session.get(Account, from_id)
            to_acc = await session.get(Account, to_id)

            from_acc.balance -= amount
            to_acc.balance += amount
            from_acc.version += 1
            to_acc.version += 1

            try:
                await session.commit()
                return {"status": "success"}
            except Exception: # IntegrityError в реальности
                await session.rollback()
                await session.refresh(from_acc)
                await session.refresh(to_acc)
                continue

        raise Exception("Max retries exceeded")

```

## 2.7. Alembic: миграции

```

In [23]: # Установка
pip install alembic

# Инициализация
alembic init alembic

# Структура:
# alembic.ini – конфигурация
# alembic/versions/ – файлы миграций
# alembic/env.py – скрипт окружения

# Создание миграции
alembic revision --autogenerate -m "Add users table"

```



```
# Применение
alembic upgrade head      # до последней
alembic upgrade +1        # на одну вперед
alembic downgrade -1      # на одну назад
alembic downgrade base    # до начала
```

## Пример миграции:

```
In [24]: """Add users table

Revision ID: a1b2c3d4
Revises:
Create Date: 2024-01-15 10:00:00.000000
"""

from alembic import op
import sqlalchemy as sa

revision = 'a1b2c3d4'
down_revision = None

def upgrade():
    op.create_table(
        'users',
        sa.Column('id', sa.Integer(), nullable=False),
        sa.Column('name', sa.String(), nullable=False),
        sa.Column('email', sa.String(), nullable=False),
        sa.PrimaryKeyConstraint('id'),
        sa.UniqueConstraint('email')
    )

def downgrade():
    op.drop_table('users')
```

## Expand/Contract для zero-downtime:

```
In [25]: War 1 (Expand):
          ALTER TABLE users ADD COLUMN login VARCHAR(255);
          # Код пишет в username и login

War 2 (Migrate):
          UPDATE users SET login = username WHERE login IS NULL;

War 3 (Contract):
```

```
# Код читает из login
ALTER TABLE users DROP COLUMN username;
```

## 2.8. Redis: кэш и rate limiting

```
In [26]: import asyncio

# Имитация Redis для семинара (без реального сервера)
class FakeRedis:
    def __init__(self):
        self._data = {}
        self._ttl = {}

    async def get(self, key: str):
        if key in self._ttl and self._ttl[key] < asyncio.get_event_loop().time():
            del self._data[key]
            del self._ttl[key]
            return None
        return self._data.get(key)

    async def setex(self, key: str, seconds: int, value: str):
        self._data[key] = value
        self._ttl[key] = asyncio.get_event_loop().time() + seconds

    async def incr(self, key: str):
        self._data[key] = self._data.get(key, 0) + 1
        return self._data[key]

    async def expire(self, key: str, seconds: int):
        self._ttl[key] = asyncio.get_event_loop().time() + seconds

redis_client = FakeRedis()

# Кэширование
async def cached_predict(input_hash: str):
    cached = await redis_client.get(f"pred:{input_hash}")
    if cached:
        return {"cached": True, "result": cached}

# Имитация ML-предсказания
await asyncio.sleep(0.5)
result = f"prediction_for_{input_hash}"
await redis_client.setex(f"pred:{input_hash}", 3600, result)
return {"cached": False, "result": result}
```

```

# Rate Limiting (sliding window)
async def rate_limit_check(client_id: str, limit: int = 100, window: int = 60):
    key = f"rate_limit:{client_id}"
    current = await redis_client.get(key)
    if current and int(current) >= limit:
        raise Exception("Too many requests") # HTTPException(429) в реальности

    await redis_client.incr(key)
    await redis_client.expire(key, window)

async def demo_redis():
    # Кэш
    r1 = await cached_predict("abc123")
    print(f"First: {r1}")
    r2 = await cached_predict("abc123")
    print(f"Second (cached): {r2}")

    # Rate limit
    for i in range(5):
        await rate_limit_check("client_1", limit=3, window=60)
        print(f"Request {i+1}: OK")

    try:
        await rate_limit_check("client_1", limit=3, window=60)
    except Exception as e:
        print(f"Request 6: {e}")

asyncio.run(demo_redis())

```

## Задания для самостоятельной работы

### Задание 1. Базовая зависимость с Depends

Напишите функцию-зависимость `get_pagination`, которая принимает query-параметры `skip` и `limit` (с валидацией: `skip >= 0`, `limit` от 1 до 100). Используйте её в двух endpoint'ах.

```

In [27]: from fastapi import FastAPI, Depends, Query

app = FastAPI()

async def get_pagination(
    skip: int = Query(ge=0, default=0),
    limit: int = Query(ge=1, le=100, default=10)

```

```

):
    return {"skip": skip, "limit": limit}

@app.get("/items/")
async def list_items(pagination: dict = Depends(get_pagination)):
    return {"items": [f"item_{i}" for i in range(pagination["skip"], pagination
["skip"] + pagination["limit"])],
            "pagination": pagination}

@app.get("/users/")
async def list_users(pagination: dict = Depends(get_pagination)):
    return {"users": [f"user_{i}" for i in range(pagination["skip"], pagination
["skip"] + pagination["limit"])],
            "pagination": pagination}

# uvicorn seminar_56:app --port 8000
# curl "http://localhost:8000/items/?skip=0&limit=3"
# curl "http://localhost:8000/users/?skip=10&limit=5"
# curl "http://localhost:8000/items/?skip=-1" # 422 Unprocessable Entity

```

## Задание 2. Вложенные зависимости

Напишите три уровня вложенных зависимостей: `get_token` -> `get_current_user` -> `get_user_permissions`. Endpoint должен возвращать права пользователя.

```

In [28]: from fastapi import FastAPI, Depends, HTTPException, Header

app = FastAPI()

async def get_token(x_token: str = Header()):
    if not x_token:
        raise HTTPException(status_code=401, detail="Missing token")
    return x_token

async def get_current_user(token: str = Depends(get_token)):
    # Имитация проверки токена
    users = {"abc123": {"id": 1, "name": "Alice"}, "def456": {"id": 2, "name": "Bob"}}
    if token not in users:
        raise HTTPException(status_code=401, detail="Invalid token")
    return users[token]

async def get_user_permissions(user: dict = Depends(get_current_user)):
    permissions = {

```

```

        1: ["read", "write", "admin"],
        2: ["read"]
    }
    return {"user": user, "permissions": permissions.get(user["id"], [])}

@app.get("/permissions/")
async def read_permissions(perms: dict = Depends(get_user_permissions)):
    return perms

# curl -H "X-Token: abc123" http://localhost:8000/permissions/
# curl -H "X-Token: def456" http://localhost:8000/permissions/
# curl -H "X-Token: invalid" http://localhost:8000/permissions/ # 401

```

### Задание 3. Зависимость с yield (setup/teardown)

Напишите зависимость `get_timer`, которая при входе запоминает время, а при выходе печатает, сколько прошло. Используйте её в endpoint, который делает `asyncio.sleep(0.5)`.

```

In [29]: from fastapi import FastAPI, Depends
import time
import asyncio

app = FastAPI()

async def get_timer():
    start = time.perf_counter()
    print(f"START at {start:.4f}")
    yield {"start": start}
    elapsed = time.perf_counter() - start
    print(f"DONE after {elapsed:.4f}s")

@app.get("/slow/")
async def slow_endpoint(timer: dict = Depends(get_timer)):
    await asyncio.sleep(0.5)
    return {"message": "completed", "timer": timer}

# uvicorn seminar_56:app --port 8000
# curl http://localhost:8000/slow/
# В консоли: START -> DONE after ~0.5s

```

### Задание 4. Repository + Unit of Work

Создайте `InMemoryUserRepository` и `InMemoryUnitOfWork`. Endpoint `POST /users/` должен создавать пользователя через `Repository`, а `UoW` должен "коммитить" изменения. Продемонстрируйте, что при исключении в endpoint происходит rollback.

```
In [30]: from fastapi import FastAPI, Depends, HTTPException
from pydantic import BaseModel
from typing import Optional

app = FastAPI()

class UserCreate(BaseModel):
    name: str
    email: str

class User(BaseModel):
    id: int
    name: str
    email: str

# --- Repository ---
class InMemoryUserRepository:
    def __init__(self):
        self._users: dict[int, User] = {}
        self._counter = 1

    async def create(self, user_data: UserCreate) -> User:
        user = User(id=self._counter, name=user_data.name, email=user_data.email)
        self._users[user.id] = user
        self._counter += 1
        return user

    async def get_by_id(self, user_id: int) -> Optional[User]:
        return self._users.get(user_id)

# --- Unit of Work ---
class InMemoryUnitOfWork:
    def __init__(self):
        self.repo = InMemoryUserRepository()
        self._committed = False
        self._rolled_back = False

    async def commit(self):
        self._committed = True
        print("UoW: COMMIT")
```

```

    async def rollback(self):
        self._rolled_back = True
        print("UoW: ROLLBACK")

# --- Зависимости ---
async def get_uow():
    uow = InMemoryUnitOfWork()
    try:
        yield uow
        await uow.commit()
    except Exception:
        await uow.rollback()
        raise

# --- Endpoint ---
@app.post("/users/")
async def create_user(
    user_data: UserCreate,
    uow: InMemoryUnitOfWork = Depends(get_uow)
):
    user = await uow.repo.create(user_data)
    return {"user": user, "committed": uow._committed, "rolled_back": uow._rolled_back}

@app.post("/users-fail/")
async def create_user_fail(
    user_data: UserCreate,
    uow: InMemoryUnitOfWork = Depends(get_uow)
):
    user = await uow.repo.create(user_data)
    raise ValueError("Simulated error") # вызовет ROLLBACK

# uvicorn seminar_56:app --port 8000
# curl -X POST http://localhost:8000/users/ -H "Content-Type: application/json" -d '{"name":"Alice","email":"a@b.com"}'
# curl -X POST http://localhost:8000/users-fail/ -H "Content-Type: application/json" -d '{"name":"Bob","email":"b@c.com"}'

```

## Задание 5. Тестирование с `dependency_overrides`

Напишите приложение с зависимостью `get_settings`, которая возвращает конфигурацию. В тестах переопределите её на фейковую. Используйте `TestClient`.

```

In [31]: from fastapi import FastAPI, Depends
from fastapi.testclient import TestClient

app = FastAPI()

class Settings:
    def __init__(self):
        self.database_url = "postgresql://real"
        self.debug = False

def get_settings():
    return Settings()

@app.get("/config/")
async def read_config(settings: Settings = Depends(get_settings)):
    return {"db": settings.database_url, "debug": settings.debug}

# --- Тесты ---
class FakeSettings:
    database_url = "sqlite:///memory:"
    debug = True

app.dependency_overrides[get_settings] = lambda: FakeSettings()

client = TestClient(app)

def test_read_config():
    response = client.get("/config/")
    assert response.status_code == 200
    data = response.json()
    assert data["db"] == "sqlite:///memory:"
    assert data["debug"] is True
    print(f"Test passed: {data}")

test_read_config()

# Очистка
app.dependency_overrides.clear()

```

## Задание 6. SQLAlchemy 2.0: CRUD

Создайте модель `Product` (id, name, price, quantity) и напишите функции для создания, чтения, обновления и удаления. Используйте `aiosqlite` для асинхронной SQLite.



```
In [32]: from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession, async_sessionmaker

from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
from sqlalchemy import select, update, delete

class Base(DeclarativeBase):
    pass

class Product(Base):
    __tablename__ = "products"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str]
    price: Mapped[float]
    quantity: Mapped[int]

DATABASE_URL = "sqlite+aiosqlite:///./products.db"
engine = create_async_engine(DATABASE_URL, echo=False)
AsyncSessionLocal = async_sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

async def init_products_db():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)

async def create_product(session: AsyncSession, name: str, price: float, quantity: int):
    product = Product(name=name, price=price, quantity=quantity)
    session.add(product)
    await session.commit()
    await session.refresh(product)
    return product

async def get_product(session: AsyncSession, product_id: int):
    result = await session.execute(select(Product).where(Product.id == product_id))
    return result.scalar_one_or_none()

async def update_product(session: AsyncSession, product_id: int, **kwargs):
    await session.execute(update(Product).where(Product.id == product_id).values(**kwargs))
    await session.commit()

async def delete_product(session: AsyncSession, product_id: int):
    await session.execute(delete(Product).where(Product.id == product_id))
    await session.commit()
```

```

async def demo_products():
    await init_products_db()
    async with AsyncSessionLocal() as session:
        p1 = await create_product(session, "Laptop", 999.99, 10)
        print(f"Created: {p1.id}, {p1.name}, ${p1.price}")

        found = await get_product(session, p1.id)
        print(f"Found: {found.name}, qty={found.quantity}")

        await update_product(session, p1.id, quantity=5)
        updated = await get_product(session, p1.id)
        print(f"Updated: qty={updated.quantity}")

        await delete_product(session, p1.id)
        deleted = await get_product(session, p1.id)
        print(f"Deleted: {deleted}")

import asyncio
asyncio.run(demo_products())

```

## Задание 7. FastAPI + SQLAlchemy: сессия на запрос

Создайте FastAPI-приложение с зависимостью `get_db`, которая создает `AsyncSession` на каждый запрос. Реализуйте endpoint'ы для создания и получения продукта.

```

In [33]: from fastapi import FastAPI, Depends, HTTPException
        from sqlalchemy.ext.asyncio import AsyncSession

        app = FastAPI()

        async def get_db():
            async with AsyncSessionLocal() as session:
                yield session

        @app.post("/products/")
        async def create_product_endpoint(
            name: str,
            price: float,
            quantity: int,
            db: AsyncSession = Depends(get_db)
        ):
            product = Product(name=name, price=price, quantity=quantity)
            db.add(product)
            await db.commit()

```

```

    await db.refresh(product)
    return {"id": product.id, "name": product.name, "price": product.price, "quantity": product.quantity}

@app.get("/products/{product_id}")
async def get_product_endpoint(product_id: int, db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(Product).where(Product.id == product_id))
    product = result.scalar_one_or_none()
    if not product:
        raise HTTPException(status_code=404, detail="Product not found")
    return {"id": product.id, "name": product.name, "price": product.price, "quantity": product.quantity}

# uvicorn seminar_56:app --port 8000
# curl -X POST "http://localhost:8000/products/?name=Phone&price=699.99&quantity=50"
# curl http://localhost:8000/products/1

```

## Задание 8. Retry с exponential backoff и jitter

Напишите декоратор `retry_with_backoff`, который повторяет асинхронную функцию при исключении `ConnectionError`. Используйте exponential backoff с jitter. Протестируйте на функции, которая падает 2 раза, а потом успешно завершается.

```

In [34]: import random
import asyncio
import functools

def retry_with_backoff(max_retries=3, base_delay=0.1):
    def decorator(func):
        @functools.wraps(func)
        async def wrapper(*args, **kwargs):
            for attempt in range(max_retries):
                try:
                    return await func(*args, **kwargs)
                except ConnectionError as e:
                    if attempt == max_retries - 1:
                        raise
                    delay = base_delay * (2 ** attempt) + random.uniform(0, 0.1)
                    print(f"Attempt {attempt + 1} failed: {e}. Retrying in {delay:.3f}s...")
                    await asyncio.sleep(delay)
            return None
    return decorator

```

```

        return wrapper
    return decorator

# Тестовая функция
_call_count = 0

@retry_with_backoff(max_retries=3, base_delay=0.1)
async def flaky_connect():
    global _call_count
    _call_count += 1
    if _call_count < 3:
        raise ConnectionError(f"Failed attempt {_call_count}")
    return {"status": "connected", "attempts": _call_count}

async def demo_retry():
    global _call_count
    _call_count = 0
    result = await flaky_connect()
    print(f"Result: {result}")

asyncio.run(demo_retry())

```

## Задание 9. Fake-репозиторий для тестов

Создайте `FakeProductRepository`, который имитирует работу с БД в памяти. Напишите тесты для endpoint'ов из задания 7, используя `dependency_overrides` и `TestClient`.

```

In [35]: from fastapi import FastAPI, Depends, HTTPException
        from fastapi.testclient import TestClient
        from pydantic import BaseModel
        from typing import Optional

        app = FastAPI()

        class ProductCreate(BaseModel):
            name: str
            price: float
            quantity: int

        class Product(BaseModel):
            id: int
            name: str
            price: float
            quantity: int

```

```

# --- Fake Repository ---
class FakeProductRepository:
    def __init__(self):
        self._products = {}
        self._counter = 1

    async def create(self, data: ProductCreate) -> Product:
        product = Product(id=self._counter, **data.model_dump())
        self._products[product.id] = product
        self._counter += 1
        return product

    async def get_by_id(self, product_id: int) -> Optional[Product]:
        return self._products.get(product_id)

fake_repo = FakeProductRepository()

async def get_product_repo():
    return fake_repo

@app.post("/products/")
async def create_product(data: ProductCreate, repo = Depends(get_product_repo)):
    product = await repo.create(data)
    return product

@app.get("/products/{product_id}")
async def get_product(product_id: int, repo = Depends(get_product_repo)):
    product = await repo.get_by_id(product_id)
    if not product:
        raise HTTPException(status_code=404, detail="Not found")
    return product

# --- Тесты ---
def test_create_and_get_product():
    app.dependency_overrides[get_product_repo] = lambda: FakeProductRepository()

    client = TestClient(app)

    # Создание
    response = client.post("/products/", json={"name": "Tablet", "price": 499.99,
"quantity": 20})
    assert response.status_code == 200
    data = response.json()
    assert data["name"] == "Tablet"
    product_id = data["id"]

```

```

# Получение
response = client.get(f"/products/{product_id}")
assert response.status_code == 200
assert response.json()["price"] == 499.99

# Не найдено
response = client.get("/products/999")
assert response.status_code == 404

print("All tests passed!")
app.dependency_overrides.clear()

test_create_and_get_product()

```

## Задание 10. Redis: кэширование с TTL

Напишите функцию `get_or_compute`, которая сначала проверяет кэш (FakeRedis), и если данных нет — вычисляет результат (имитация: `asyncio.sleep(1)`), сохраняет в кэш на 5 секунд и возвращает. Продемонстрируйте, что второй вызов с тем же ключом возвращает результат мгновенно.

```

In [36]: import asyncio

class FakeRedis:
    def __init__(self):
        self._data = {}
        self._ttl = {}

    async def get(self, key: str):
        now = asyncio.get_event_loop().time()
        if key in self._ttl and self._ttl[key] < now:
            del self._data[key]
            del self._ttl[key]
            return None
        return self._data.get(key)

    async def setex(self, key: str, seconds: int, value: str):
        self._data[key] = value
        self._ttl[key] = asyncio.get_event_loop().time() + seconds

redis = FakeRedis()

async def expensive_computation(key: str) -> str:

```

```

await asyncio.sleep(1) # имитация тяжелой работы
return f"computed_result_for_{key}"

async def get_or_compute(key: str) -> dict:
    cached = await redis.get(f"cache:{key}")
    if cached:
        return {"cached": True, "result": cached, "time": "instant"}

    result = await expensive_computation(key)
    await redis.setex(f"cache:{key}", 5, result)
    return {"cached": False, "result": result, "time": "1s"}

async def demo_cache():
    import time

    # Первый вызов — вычисление
    start = time.perf_counter()
    r1 = await get_or_compute("user_123")
    t1 = time.perf_counter() - start
    print(f"First call: {r1} (took {t1:.2f}s)")

    # Второй вызов — из кэша
    start = time.perf_counter()
    r2 = await get_or_compute("user_123")
    t2 = time.perf_counter() - start
    print(f"Second call: {r2} (took {t2:.2f}s)")

asyncio.run(demo_cache())

```

## Задание 11. Rate Limiting через Redis

Напишите middleware (или зависимость) `rate_limit`, которая ограничивает количество запросов от одного клиента до 5 в минуту. Используйте FakeRedis с ключом `rate:{client_id}`.

```

In [37]: from fastapi import FastAPI, Depends, HTTPException, Request

app = FastAPI()

class FakeRedis:
    def __init__(self):
        self._data = {}
        self._ttl = {}

```

```

async def get(self, key: str):
    now = asyncio.get_event_loop().time()
    if key in self._ttl and self._ttl[key] < now:
        del self._data[key]
        del self._ttl[key]
        return None
    return self._data.get(key)

async def incr(self, key: str):
    self._data[key] = self._data.get(key, 0) + 1
    return self._data[key]

async def expire(self, key: str, seconds: int):
    self._ttl[key] = asyncio.get_event_loop().time() + seconds

redis = FakeRedis()

async def rate_limit(request: Request, limit: int = 5, window: int = 60):
    client_id = request.headers.get("X-Client-ID", "anonymous")
    key = f"rate:{client_id}"

    current = await redis.get(key)
    if current and int(current) >= limit:
        raise HTTPException(status_code=429, detail="Too many requests")

    count = await redis.incr(key)
    if count == 1:
        await redis.expire(key, window)

    return {"client": client_id, "requests": count, "limit": limit}

@app.get("/api/")
async def api_endpoint(rate_info: dict = Depends(rate_limit)):
    return {"message": "Success", "rate": rate_info}

# uvicorn seminar_56:app --port 8000
# for i in {1..7}; do curl -H "X-Client-ID: user1" http://localhost:8000/api/; ec
ho; done
# Первые 5 — 200, 6-7 — 429

```

## Задание 12. Полноценный endpoint с Repository + UoW + DI

Создайте endpoint `POST /orders/` для создания заказа. Используйте:

- `OrderRepository` (in-memory) для работы с заказами.



- `ProductRepository` (in-memory) для проверки наличия товара.
- `UnitOfWork` для атомарности.
- `Depends` для внедрения всех зависимостей.

```
In [38]: from fastapi import FastAPI, Depends, HTTPException
from pydantic import BaseModel
from typing import Optional, List

app = FastAPI()

class OrderItem(BaseModel):
    product_id: int
    quantity: int

class OrderCreate(BaseModel):
    customer_name: str
    items: List[OrderItem]

class Order(BaseModel):
    id: int
    customer_name: str
    items: List[OrderItem]
    total: float

# --- Repositories ---
class ProductRepository:
    def __init__(self):
        self._products = {
            1: {"id": 1, "name": "Laptop", "price": 999.99, "stock": 10},
            2: {"id": 2, "name": "Mouse", "price": 29.99, "stock": 50},
        }

    async def get_by_id(self, product_id: int):
        return self._products.get(product_id)

    async def reserve(self, product_id: int, quantity: int) -> bool:
        product = self._products.get(product_id)
        if not product or product["stock"] < quantity:
            return False
        product["stock"] -= quantity
        return True

class OrderRepository:
```

```

        self._counter = 1

    async def create(self, order_data: OrderCreate, total: float) -> Order:
        order = Order(
            id=self._counter,
            customer_name=order_data.customer_name,
            items=order_data.items,
            total=total
        )
        self._orders[order.id] = order
        self._counter += 1
        return order

# --- Unit of Work ---
class OrderUnitOfWork:
    def __init__(self):
        self.products = ProductRepository()
        self.orders = OrderRepository()
        self._committed = False

    async def commit(self):
        self._committed = True

    async def rollback(self):
        # В in-memory откат не нужен, но в реальности – восстановление stock
        pass

# --- Зависимости ---
async def get_uow():
    uow = OrderUnitOfWork()
    try:
        yield uow
        await uow.commit()
    except Exception:
        await uow.rollback()
        raise

# --- Endpoint ---
@app.post("/orders/")
async def create_order(order_data: OrderCreate, uow: OrderUnitOfWork = Depends(get_uow)):
    total = 0.0

    for item in order_data.items:
        product = await uow.products.get_by_id(item.product_id)
        if not product:

```

```

        raise HTTPException(status_code=404, detail=f"Product {item.product_id} not found")

    reserved = await uow.products.reserve(item.product_id, item.quantity)
    if not reserved:
        raise HTTPException(status_code=400, detail=f"Insufficient stock for product {item.product_id}")

    total += product["price"] * item.quantity

    order = await uow.orders.create(order_data, total)
    return {"order": order, "committed": uow._committed}

# uvicorn seminar_56:app --port 8000
# curl -X POST http://localhost:8000/orders/ -H "Content-Type: application/json" \
#   -d '{"customer_name":"Alice","items":[{"product_id":1,"quantity":1},{"product_id":2,"quantity":2}]}'
# curl -X POST http://localhost:8000/orders/ -H "Content-Type: application/json" \
#   -d '{"customer_name":"Bob","items":[{"product_id":1,"quantity":20}]}' # 400
- недостаточно stock

```

## Итоговая сводка

| Концепция             | Модуль | Ключевой API / Паттерн                                     |
|-----------------------|--------|------------------------------------------------------------|
| IoC                   | 5      | Инфраструктура предоставляет себя бизнес-логике            |
| DI                    | 5      | Зависимости передаются извне                               |
| Depends               | 5      | <code>Depends(callable)</code> , автоматическое разрешение |
| Вложенные зависимости | 5      | Рекурсивное дерево, кэширование                            |
| yield в DI            | 5      | Генераторы для setup/teardown                              |
| Repository            | 5      | Абстракция над источником данных, Protocol                 |
| Unit of Work          | 5      | Атомарная группа операций                                  |
| dependency_overrides  | 5      | Подмена зависимостей в тестах                              |
| Test Double           | 5      | Fake, Mock, Stub, Spy                                      |

| Концепция                       | Модуль | Ключевой API / Паттерн                                                                        |
|---------------------------------|--------|-----------------------------------------------------------------------------------------------|
| ACID                            | 6      | Атомарность, согласованность, изоляция, долговечность                                         |
| Уровни изоляции                 | 6      | READ COMMITTED, REPEATABLE READ, SERIALIZABLE                                                 |
| Феномены                        | 6      | Dirty Read, Non-Repeatable Read, Phantom Read, Lost Update                                    |
| B-Tree                          | 6      | $O(\log_B n)$ поиск, фактор ветвления                                                         |
| SQLAlchemy 2.0                  | 6      | <code>select()</code> , <code>insert()</code> , <code>update()</code> , <code>delete()</code> |
| <code>async_sessionmaker</code> | 6      | <code>expire_on_commit=False</code> , <code>autoflush=False</code>                            |
| Сессия на запрос                | 6      | <code>get_db()</code> с <code>yield</code> и <code>AsyncSession</code>                        |
| Connection Pool                 | 6      | <code>pool_size</code> , <code>max_overflow</code> , <code>pool_timeout</code>                |
| Deadlock                        | 6      | Retry с exponential backoff + jitter                                                          |
| Оптимистичные блокировки        | 6      | Версионирование, <code>WHERE id=? AND version=?</code>                                        |
| Alembic                         | 6      | <code>revision --autogenerate</code> , <code>upgrade head</code>                              |
| Expand/Contract                 | 6      | Zero-downtime миграции                                                                        |
| Redis                           | 6      | Кэш с TTL, rate limiting, pub/sub                                                             |
| Векторные БД                    | 6      | Embeddings, similarity search, HNSW                                                           |

## Чек-лист для самопроверки

- ☐ Я могу объяснить разницу между IoC и DI.
- ☐ Я понимаю, как FastAPI разрешает дерево зависимостей.
- ☐ Я умею использовать `yield` в зависимостях для управления ресурсами.
- ☐ Я могу написать Repository и Unit of Work с нуля.
- ☐ Я умею тестировать endpoint'ы с `dependency_overrides` .
- ☐ Я понимаю все 4 свойства ACID и могу привести примеры.
- ☐ Я знаю разницу между уровнями изоляции и когда какой использовать.

- ☐ Я могу настроить SQLAlchemy 2.0 для асинхронной работы.
- ☐ Я понимаю, почему `expire_on_commit=False` важен для async.
- ☐ Я умею обрабатывать deadlock'и с retry-логикой.
- ☐ Я знаю, как работает Alembic и когда проверять autogenerate вручную.
- ☐ Я понимаю паттерн Expand/Contract для zero-downtime деплоя.
- ☐ Я могу реализовать кэширование и rate limiting через Redis.